

Buffalo Curve Analytics

Code Explainer — File by File

What each module does, and how a request flows through it

University of Naples Federico II

MSc Data Science

Supervisor: Prof. Schiano Lo Moriello

October 2026

1. File Map

Ten files carry the system. Everything else is generated output, sample data or deployment configuration.

File	Lines	Responsibility
pipeline/vector_extract.py	686	Reads curves from the PDF vector layer and calibrates both axes
pipeline/pipeline.py	1302	Orchestration, integration, bounded calibration, CSV and images
pipeline/settings.py	159	Every threshold and constant; the single source of truth
app.py	438	Flask backend — upload, job registry, polling, downloads
frontend/index.html	1384	Single-file UI, original and extracted charts side by side
tests/verify_against_pdf.py	233	Independent re-measurement of the curves at 600 DPI
tests/verify_metadata.py	176	Checks header fields against the PDF text layer
demo/make_demos.py	282	Generates demo reports with ground truth
demo/score_demos.py	132	Scores extraction against that ground truth
Dockerfile, render.yaml	35, 15	Container build and one-click deployment

2. pipeline/vector_extract.py

The module that replaced pixel masking. It never rasterises; it asks the PDF what it drew.

has_vector_curves(page)

Gate function. Scans the drawing list for at least 20 strokes passing the blue or green test, and returns True if the page carries a real vector layer. A scanned or flattened PDF fails here and the raster path in pipeline.py is used instead.

_is_blue(c) and _is_green(c)

The only thing separating flow from pressure. Each is a three-part test on the stroke's DeviceRGB colour, with the limits held in settings.py:

```
def _is_blue(c):
    return (c[2] > settings.BLUE_MIN_B # blue > 0.55
            and c[0] < settings.BLUE_MAX_R # red < 0.45
            and c[1] < settings.BLUE_MAX_G) # green < 0.45
```

A stroke failing all three tests is skipped entirely. There is no nearest-match fallback, so a colour change in the source produces an empty result rather than a quietly wrong one.

_y_calibration(...) and _tick_value_range(...)

Builds the vertical ruler. The axis tick labels sit at known heights, so a least-squares fit through those (y, value) pairs converts any coordinate to kg/min. Each chart is fitted on its own; the residual is carried into the result and stays below 0.004.

_x_calibration(...), _tick_x(...), _frame_x(...)

Builds the horizontal ruler without reading a single tick label. `_tick_x` finds the short vertical marks along the lower frame and `_x_calibration` detects their uniform pitch — 21 marks at 13.2 pt on the sample report. `_frame_x` locates the plot box by testing individual segments rather than path bounding boxes, which is what makes detection reliable when a chart overruns the page midpoint.

_collect_points(...) and _points_to_series(...)

`_collect_points` gathers every segment endpoint inside the chart box that passes the colour test — 86 points for a typical curve. `_points_to_series` sorts them by x, applies both calibrations and returns the (time, value) arrays.

`_parse_header(...)` and `extract_page(page, n)`

`_parse_header` reads animal ID, date, time and the printed milk weight from the text layer. `extract_page` is the entry point: it slices the page into the 4×2 grid, runs the steps above per cell, and returns a list of `ChartData` records carrying the series, the calibration, the crop rectangle and the `reaches_axis_end` flag.

3. pipeline/pipeline.py

Orchestrates extraction and owns everything that happens after the curve has been measured.

`run_pipeline(input_path, out_dir)`

Entry point used by both the web app and the command line. Opens the PDF, asks `has_vector_curves` whether the vector path applies, and dispatches to `_run_vector` or `_run_cv` accordingly.

`_calibrate_to_printed_milk(f_t, f_v, milk_kg)`

The most consequential function in the file, and the one most worth reading closely. The AMS prints its own measured weight on every chart. The drawn line has width, so the true curve lies somewhere inside the ink and a small scale legitimately moves within that band.

The bound is the ink itself. Half the pen width is the most any single reading can be out by, so over a session of length T it accounts for at most that much area:

```
duration = float(t[-1] - t[0])
allowed = (settings.PEN_HALF_WIDTH_KG_MIN * duration
           * settings.CALIBRATION_TOLERANCE)
gap      = abs(milk_kg - area)

if gap > allowed:
    # Beyond ink ambiguity - leave it alone.
    return v, scale, False, "exceeds_pen_width"

return v * scale, scale, True, None
```

A wider gap is not ink ambiguity; it is milk the chart never drew, which happens when the session ran past the right edge of the time axis. Those curves are returned exactly as measured, and the caller records `milk_pct_of_printed` so the chart reports the figure it actually reaches. On the sample report 10 of 16 charts calibrate inside the bound and 6 are refused.

`_build_long_csv(...)`

Resamples both series onto an even grid at `CSV_TIME_STEP_MIN` and writes the three columns. A second, tightly bounded re-fit ($\pm 0.1\%$) runs here so the number in the file integrates exactly on the sampled grid rather than only on the dense vertices.

`_series_metrics, _measure_curve_fidelity, _render_chart_image`

Peak, integral and duration per series; the chart crop used for the side-by-side comparison in the UI; and `_measure_curve_fidelity`, which produces the accuracy figure. It re-renders each chart, reads the drawn line back through an independent pixel path and reports the distance from the extracted curve, so the check grades the extraction without sharing any code with it. Sixteen charts take 1.2 seconds.

4. pipeline/settings.py

No threshold is written inline anywhere else. Changing behaviour means editing this file, which is what makes the system reproducible between runs and inspectable by a reviewer.

Constant	Value	Governs
GRID_ROWS, GRID_COLS	4, 2	Chart layout per page
BLUE_MIN_B / MAX_R / MAX_G	0.55 / 0.45 / 0.45	Which strokes count as flow
GREEN_MIN_G / MAX_R / MAX_B	0.35 / 0.45 / 0.45	Which strokes count as pressure
CSV_TIME_STEP_MIN	0.02	Row spacing in the CSV
CALIBRATE_TO_PRINTED_MILK	True	Whether to scale at all
PEN_HALF_WIDTH_KG_MIN	0.0389	The calibration bound
CALIBRATION_TOLERANCE	1.15	Headroom above that bound
FLOW_THRESHOLD_KG_MIN	0.20	The AMS's own printed cut-off
RUN_FIDELITY_CHECK	True	Scores every chart against the drawn line (1.2 s per report)

5. app.py

A small Flask service. No database, no external calls, no login.

Route	Purpose
GET /	Serves the single-page frontend
GET /build	Returns the build id and measurement rules, so a stale deployment is obvious without uploading anything
POST /upload	Accepts the PDF, starts a background job, returns its id
GET /status/<id>	Progress, per-chart payload and summary
GET /download/<id>/<file>	One CSV, path-guarded
GET /chart-image/<id>/<file>	The original chart crop
GET /download-all/<id>	Every CSV as one ZIP

`_chart_payload` shapes one chart for the browser, including `milk_uncalibrated_reason` and `milk_pct_of_printed` so a refused chart can explain itself. `_summarise` aggregates the report and reports charts `_calibrated` against `charts_measured_only`. `_safe_job_file` resolves and confines every download path, so a crafted filename cannot escape the job directory.

The job registry is a dictionary inside the process, so the supplied configuration runs one gunicorn worker with several threads. Raising the worker count requires moving that registry to shared storage first.

6. frontend/index.html

One file, no build step and no framework, so it can be opened and read directly. It polls `/status`, renders each chart beside its original crop, and computes session statistics in the browser using the thresholds served by `/build` rather than its own copies.

A chart whose calibration was refused is labelled — “ran past axis” where the session continued beyond the chart, otherwise “as measured” — so a low percentage reads as a property of the source rather than an extraction fault.

Polling stops on a completed job, with a retry guard: a status of done with an empty file list is re-pollled a few times rather than treated as a finished job, which is what previously produced the “0 CSV files generated” report.

7. Tests and Demo Data

Neither test shares code with the extractor, which is what makes them worth running.

tests/verify_against_pdf.py rasterises the report at 600 DPI — four times finer than the old pipeline ever worked at — and measures the curves again by an independent path. Agreement on the sample report is 0.0099 kg/min.

tests/verify_metadata.py re-reads every header field straight from the text layer and compares it with what the pipeline recorded.

demo/make_demos.py generates reports in the real layout, 2 pages × 8 charts, writing the exact curve coordinates to JSON first. **demo/score_demos.py** then scores extraction against that truth: 0.0037 kg/min mean error over 64 charts. Because these contain no truncated session, all 16 charts in each calibrate inside the bound.

8. How a Request Flows

1. The browser POSTs the PDF to /upload. app.py saves it, creates a job id and starts a worker thread.
2. run_pipeline opens the document and calls has_vector_curves. A real AMS report takes the vector path.
3. For each page, extract_page slices the 4 × 2 grid and, per chart, collects the coloured segments, fits both axes and parses the header.
4. Back in pipeline.py the flow series is integrated, offered to _calibrate_to_printed_milk and either scaled within the ink bound or kept as measured with a reason.
5. _build_long_csv writes the three columns; _render_chart_image saves the original crop for comparison.
6. The browser polls /status, draws each chart beside its crop, and offers the CSVs individually or as one ZIP.

Reading order for a reviewer: settings.py first, since it states every rule the system follows; then vector_extract.py for how a curve is recovered; then _calibrate_to_printed_milk for what is done with it afterwards, which is where the accuracy claim is decided.